UNIVERSIDAD TECNOLÓGICA NACIONAL

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

METODOLOGIA EN SISTEMAS 2

TRABAJO PRACTICO INTEGRADOR

Evolución Profesional de un sistema existente

Calidad, deuda técnica, refactoring y revisión

Alumnos:

Alex Nahuel Austin

Leonel Jesus Aballay

Comisión: 6

Anexos y enlaces

Recurso	Enlace
GitHub	https://github.com/AlexNahuelAustin/Metodologia-II-Trabajo-Integrador.git
Video de defensa	https://www.youtube.com/watch?v=722BKH-Q54I&feature=youtu.be

Resumen ejecutivo

Se analizó y mejoró la calidad interna de un módulo de gestión de solicitudes académicas desarrollado en

Java 21 con persistencia en H2. La versión inicial concentraba validaciones, reglas de negocio, acceso a datos y

presentación en una única clase de 300 líneas. Se identificaron ocho code smells con su evidencia, ubicación,

impacto y prioridad, y se aplicaron cinco refactorings documentados que separaron las responsabilidades en

tres capas. Una suite de catorce pruebas unitarias con dobles en memoria verificó que el comportamiento

externo se mantuvo sin regresiones. Finalmente se elaboró y aplicó un checklist de code review de quince

criterios, que dejó registradas seis observaciones de deuda pendiente con su recomendación técnica.

Descripción del sistema base

Dominio y alcance

Módulo de gestión de solicitudes académicas de una institución educativa. Permite que un estudiante registre

una solicitud, que se le asigne un responsable del área correspondiente y que esa solicitud avance por una

máquina de estados hasta cerrarse.

Actores

Actor	Rol
Estudiante	Origina la solicitud, sus datos se registran automáticamente en el primer alt
Responsable	Agente administrativo que toma la solicitud. Tiene un tope de carga
Sistema	Valida, asigna prioridad según el tipo y controla las transiciones de estado

Entidades

- Solicitud — id, legajo del estudiante, tipo, estado, prioridad, responsable asignado, fecha de alta, fecha de cierre y observaciones.
- Estudiante — legajo, nombre, apellido, email, teléfono y carrera.
- Responsable — id, nombre, área y cantidad de solicitudes asignadas.

Funcionalidades verificables

1. Registrar solicitud — valida legajo, email y tipo; crea el estudiante si no existe; controla el tope de cinco solicitudes abiertas; asigna la prioridad según el tipo.

2. Cambiar estado — respeta las transiciones válidas: pendiente → en_proceso | rechazada y en_proceso → resuelta | rechazada. Los estados resuelta y rechazada son finales. No permite pasar a en_proceso sin responsable asignado.

3. Asignar responsable — verifica que existan la solicitud y el responsable, y que este no supere las diez asignaciones.

4. Buscar — por estado, por tipo o por responsable.

5. Generar reporte — totales por estado, demora promedio de cierre y listado de solicitudes vencidas (más de treinta días abiertas).

Reglas de negocio

Regla	Valor
Maximo de solicitudes abiertas por estudiantes	5
Maximo de asignaciones por responsable	10
Dias para considerar una solicitud vencida	30
Prioridad de reincorporación/Equivalencia/Certificado	1/2/3

Tecnología

Java 21, Gradle, H2 embebida con acceso por JDBC, JUnit 5 y JaCoCo.

Evidencia de funcionamiento

La ejecución del Main demuestra los cinco casos de uso, tanto en sus caminos exitosos como en los de error.

Aplicación de SOLID y Clean Code

SRP — Responsabilidad Única

Antes: GestorDeSolicitudes tenía cuatro motivos de cambio: un cambio en las reglas de negocio, uno en el motor de base de datos, uno en el formato del reporte y uno en los mensajes al usuario obligaban todos a modificar la misma clase.

Después: cada motivo de cambio quedó aislado en su propio componente.

Componente	Motivo de cambio
ServicioDeSolicitudes	Las reglas de negocio
RepositoryDeSolicitudesH2	La tecnología de persistencia
GeneradorDeReportes	El formato del reporte
Models	La estructura de los datos

OCP — Abierto/Cerrado

Antes: agregar un estado nuevo obligaba a modificar el **if/else** anidado de **cambiarEstado**, con riesgo de romper las transiciones existentes.

Después: las transiciones válidas se expresan en un **switch** sobre el enum **EstadoSolicitud**. Agregar un estado es agregar una constante y una rama; el compilador obliga a contemplarla porque el switch sobre enum debe ser exhaustivo.

LSP — Sustitución de Liskov

Se verifica en la práctica: **ServicioDeSolicitudes** funciona idénticamente con

RepositoryDeSolicitudesH2 en producción y con **RepositoryDeSolicitudesEnMemoria** en los tests.

Ninguna implementación pide más condiciones que el contrato ni promete menos, y los catorce tests pasan

sin modificaciones en ambos casos.

ISP — Segregación de Interfaces

Se definieron tres interfaces separadas —**RepositoryDeSolicitudes**, **RepositoryDeEstudiantes** y **RepositoryDeResponsables**— en lugar de una única interfaz de persistencia. **RepositoryDeEstudiantes** declara solo dos métodos: ninguna implementación queda obligada a resolver operaciones que no le corresponden.

DIP — Inversión de Dependencias

Antes: la lógica de negocio instanciaba conexiones JDBC directamente y armaba el SQL. Dependía de la tecnología concreta.

Después: **ServicioDeSolicitudes** depende únicamente de las interfaces, que define el dominio. Las implementaciones concretas se inyectan por constructor y se ensamblan en el **Main**, que actúa como composition root.

La prueba de que el DIP funciona es la suite de tests: la lógica de negocio se ejecuta sin levantar una sola conexión a la base de datos.

Clean Code

Practica	Aplicación
Nombres que revelan intención	P, ep, res, c, lst -> eliminados: los métodos privados describen el paso (imprimirDemorarPromedio, estaVencida)
Funciones pequeñas	generarReporte de 60 lineas -> cinco métodos de menos de diez
Sin números mágicos	5, 10 ,30 -> constantes con nombre, “pendiente”, “certificado” -> enums
Sin duplicación	El mapeo del ResultSet paso de estar 3 veces a un único método mapear
Manejo explicito de errores	Catch (Exception) que imprima -> ErrorDePersistencia con contexto
Comentarios innecesarios	Los que explicaban nombrs crípticos o números mágicos desaparecieron al corregir la causa

7. Refactorings aplicados

N	Refactoring	Smell que resuelve	Principio	commit
1	Introduc Parameter Object	4 –8 parametros	-	Refactor: introduce paramter object
2	Replace Magic Number with Symbolic Constant	5 y 6 – Numeros y string mágicos	-	Refactor: reemplazar números y string mágicos
3	Extrac Class	1 , 3 y 8 – God object, duplicación y excepciones	SRP+DIP	Refactor: extract class
4	Extract Method	2 – método largo	SRP	Incluido en el commit anterior
5	Replace conditional with polymorphism	7 – condicional complejo	OCP	También incluido en el commit anterior

Evidencia antes/después

Refactoring 1 — Introduce Parameter Object

```
// ANTES
public int registrarSolicitud(String legajo, String nombre, String apellido,
String email, String telefono, String carrera,
String tipo, String obs)

// DESPUÉS
public int registrarSolicitud(DatosDeEstudiante datos, String tipo, String obs)
```

Beneficio: el compilador ahora impide cruzar los argumentos. Antes, invertir nombre y apellido compilaba sin

error.

Refactoring 2 — Constantes y enums

```
// ANTES
if (cant >= 5) { ... }
if (tipo.equals("reincorporacion")) { prio = 1; }
else if (tipo.equals("equivalencia")) { prio = 2; }
else { prio = 3; }

// DESPUÉS
if (solicitudes.contarAbiertasDe(datos.legajo())
>= MAXIMO_SOLICITUDES_ABIERTAS_POR_ESTUDIANTE) { ... }
nueva.prioridad = tipoDeSolicitud.prioridad();
```

Beneficio: la prioridad dejó de ser una cadena de condicionales y pasó a ser un dato del propio tipo.

Refactoring 5 — Replace Conditional with Polymorphism

```
// ANTES: if/else anidados de dos niveles, 45 líneas
if (actual.equals("pendiente")) {
if (nuevoEstado.equals("en_proceso")) {
if (s.idResponsable == null) { ... }
} else if (nuevoEstado.equals("rechazada")) { ... }
...
```

```
}

// DESPUÉS: guardas planas más un switch exhaustivo
private boolean esTransicionValida(EstadoSolicitud actual, EstadoSolicitud
destino) {
return switch (actual) {
case PENDIENTE  -> destino == EN_PROCESO || destino == RECHAZADA;
case EN_PROCESO -> destino == RESUELTA   || destino == RECHAZADA;
case RESUELTA, RECHAZADA -> false;
};
}
```

Beneficio: la máquina de estados completa se lee en seis líneas, y agregar un estado obliga al compilador a exigir su rama.

Comparación completa: https://github.com/AlexNahuelAustin/Prueba/compare/estado-inicial...estado-refactorizado

8. Pruebas y verificación

Estrategia

La verificación se hizo en dos niveles:

1. Comparación de salida. Antes y después de cada refactoring se ejecutó el Main y se comparó la salida de consola carácter por carácter. Esta comparación detectó una regresión real: durante el refactoring 1 se perdieron dos líneas de alta de responsables, lo que hizo fallar las asignaciones. El error se corrigió antes de commitear.

2. Suite automatizada. Catorce pruebas unitarias sobre ServicioDeSolicitudes, estructuradas con el patrón AAA (Arrange, Act, Assert) y ejecutadas contra dobles de prueba en memoria.

Casos cubiertos

Grupo	Casos
Alta	Solicitud valida, sin legajo, mail invalido, tipo inexistente, prioridad por tipo, tope de cinco abiertas
Cambio de estado	Sin responsables, con responsables, transición invalida, reapertura de cerrada, registro de fecha de cierre
Asignacion	Responsable inexistente, solicitud inexistente
Búsqueda	Filtrado por estado

Resultados

Catorce pruebas en verde. (Adjuntar captura de la salida de gradlew test.)

Cobertura

Paquete	Instrucciones	Ramas
Services	58%	66%
Models	87%	83%
Repository	0%	0%
Total	29%	54%

Interpretación: la capa de servicios, que concentra las reglas de negocio, alcanza 66% de cobertura de ramas. Los adaptadores H2 quedan en 0% porque las pruebas unitarias usan dobles en memoria por diseño; cubrirlos requiere tests de integración, que se declaran como deuda pendiente.

Un dato relevante del proceso: la cobertura de ramas subió de 30% a 54% únicamente al eliminar **GestorDeSolicitudes**, que era código muerto. El reporte de JaCoCo fue la herramienta que permitió detectarlo.

10. Conclusiones y mejoras futuras

Aprendizajes

El trabajo dejó claro que **calidad externa y calidad interna** son dimensiones independientes. La versión inicial funcionaba correctamente: pasaba todos los casos de uso y los de error. Sin embargo, era prácticamente intocable, porque cualquier cambio exigía entender trescientas líneas donde convivían cuatro responsabilidades.

El segundo aprendizaje fue sobre el orden de los refactorings. Resolver primero el God Object habilitó que los demás cayeran casi solos: una vez separadas las capas, el Extract Method y el Replace Conditional fueron consecuencia natural de la nueva estructura.

El tercero, y el más concreto, fue el valor de verificar cada paso. La comparación de salidas detectó una regresión real durante el primer refactoring. Sin esa verificación, se habría commiteado un cambio que rompía la funcionalidad.

Dificultades

- **Separar el refactoring del cambio funcional.** Al introducir H2 surgió la duda de si cambiar la persistencia alteraba el comportamiento externo. Se resolvió manteniendo H2 en ambas versiones, de modo que el refactoring modificara únicamente la estructura interna.

- **Mantener commits atómicos**. Los refactorings 3, 4 y 5 terminaron en un solo commit porque la extracción de clases arrastró consigo la extracción de métodos. Lo correcto hubiera sido commitear cada uno por separado.
- **Errores de entorno.** Un paquete creado con mayúscula inicial y la ubicación de archivos en paquetes equivocados generaron errores de compilación que consumieron tiempo.

Límites del trabajo

- Los adaptadores H2 no tienen cobertura: la suite no ejecuta SQL real.
- El servicio sigue imprimiendo por consola y devolviendo códigos de retorno, mezclando presentación con negocio.
- Las entidades mantienen campos públicos, sin encapsulamiento.

Mejoras futuras

Prioridad	Mejora
Alta	Test de integración sobre H2 en memoria para cubrir los repositorios
Alta	Reemplazar los códigos de retorno (-1, false) por un tipo Resultado con el motivo del rechazo
Media	Encapsular las entidades y mover a ellas el comportamiento que les corresponde (solicitud.cerrar())
Media	Unificar la nomenclatura de las interfaces de repositorio
Baja	Validacion de mail con expresión regular

Relación con los contenidos de la asignatura

El trabajo integró las tres unidades: el diagnóstico aplicó los conceptos de calidad interna/externa, deuda técnica y code smells; el refactoring aplicó SOLID, Clean Code y el catálogo de Fowler; y la verificación aplicó el patrón AAA, los dobles de prueba y el análisis de cobertura de ramas vistos en la unidad de verificación y validación.

Tags del repositorio

estado-inicial · etapa-diagnostico · etapa-refactoring · etapa-pruebas · etapa-docs · estado-refactorizado

Declaración de uso de inteligencia artificial

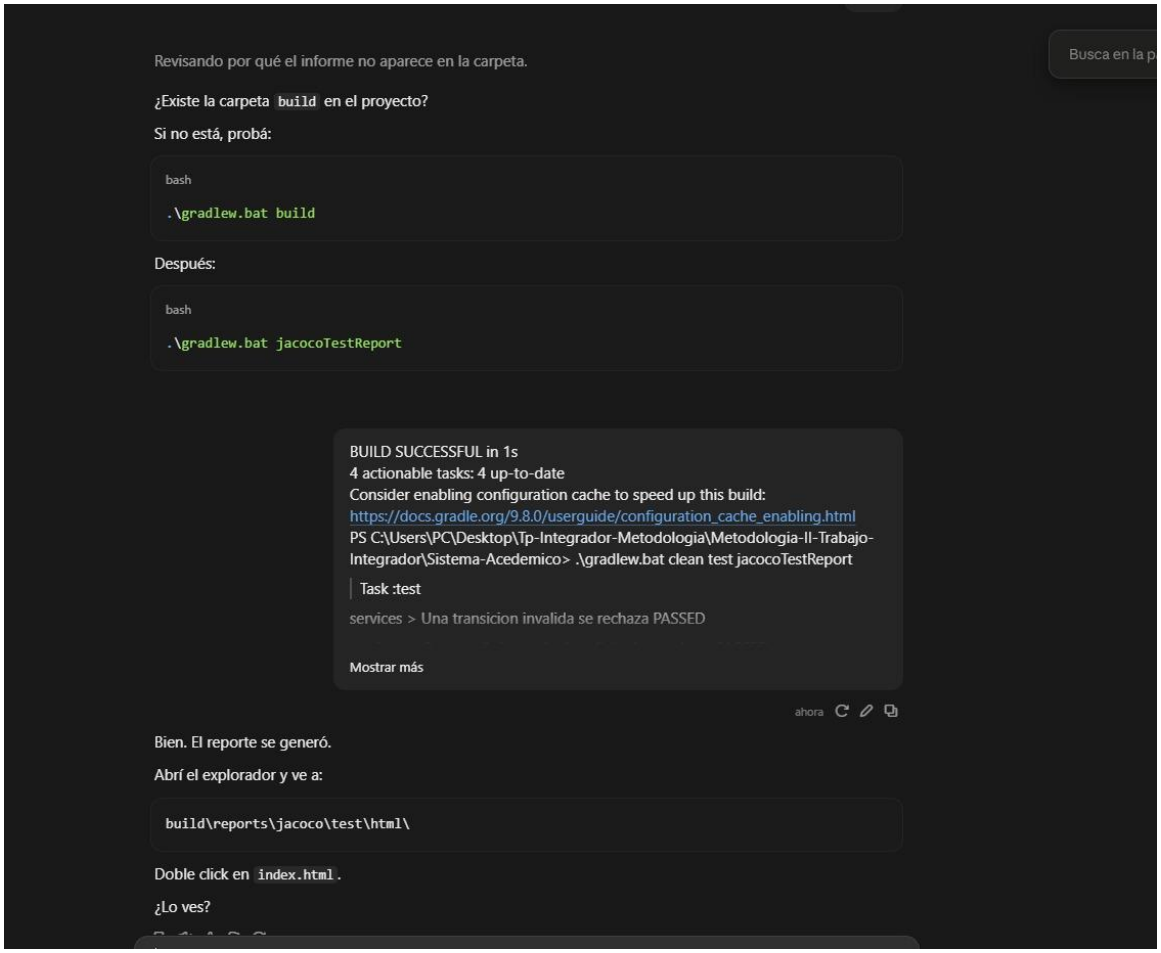
Herramienta utilizada: Claude (Anthropic)

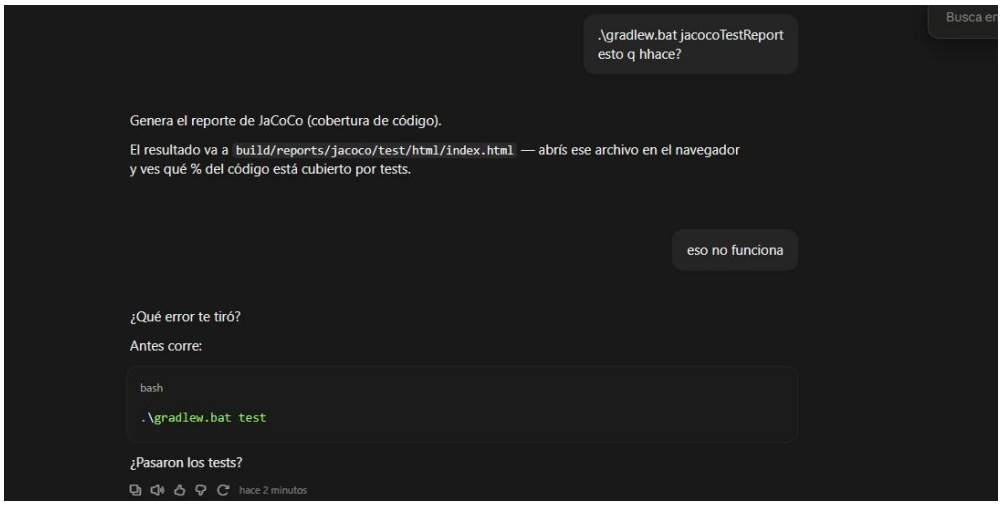
Finalidad: asistencia conceptual sobre code smells, principios SOLID y catálogo de refactorings; generación del sistema base con problemas de diseño deliberados para ser diagnosticados; apoyo en la redacción del informe y revisión del código producido.

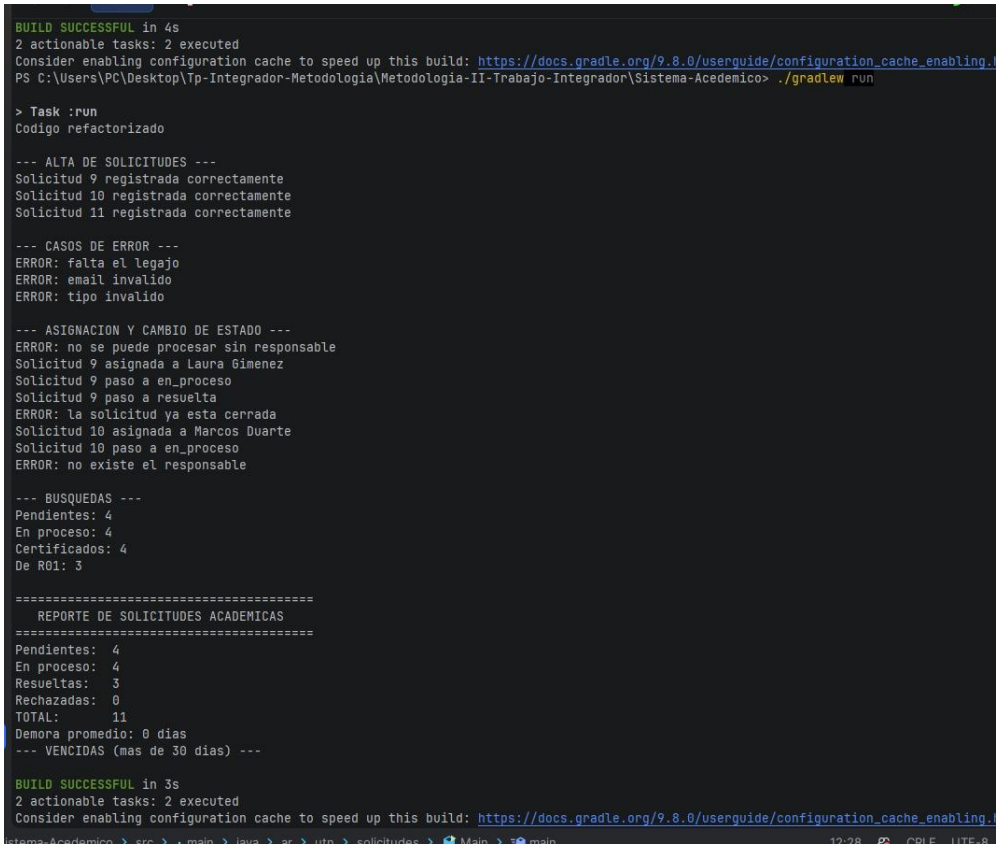
Grado de intervención: el intercambio completo se adjunta en el anexo. El alumno comprende y puede justificar cada decisión técnica documentada.

Evidencias adjuntas

- Captura de saldia del Main (estado inicial)
- Captura de las tablas de H2 con los datos cargados
- Captura del gradlew test con las catorce pruebas en verde
- Captura del reporte JaCoCo
- Captura del historial de commits con los tags
- Transcripcion del intercambio de la IA




Busca en
.\gradlew.bat jacocoTestReport
esto q hhace?
Genera el reporte de JaCoCo (cobertura de código).
El resultado va a build/reports/jacoco/test/html/index.html — abrís ese archivo en el navegador y ves qué % del código está cubierto por tests.
eso no funciona
¿Qué error te tiró?
Antes corre:
bash
.\gradlew.bat test
¿Pasaron los tests?
hace 2 minutos


BUILD SUCCESSFUL in 4s
2 actionable tasks: 2 executed
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.8.0/userguide/configuration_cache_enabling.
PS C:\Users\PC\Desktop\Tp-Integrador-Metodologia\Metodologia-II-Trabajo-Integrador\Sistema-Acedemico> ./gradlew run
> Task :run
Codigo refactorizado
--- ALTA DE SOLICITUDES ---
Solicitud 9 registrada correctamente
Solicitud 10 registrada correctamente
Solicitud 11 registrada correctamente
--- CASOS DE ERROR ---
ERROR: falta el legajo
ERROR: email invalido
ERROR: tipo invalido
--- ASIGNACION Y CAMBIO DE ESTADO ---
ERROR: no se puede procesar sin responsable
Solicitud 9 asignada a Laura Gimenez
Solicitud 9 paso a en_proceso
Solicitud 9 paso a resuelta
ERROR: la solicitud ya esta cerrada
Solicitud 10 asignada a Marcos Duarte
Solicitud 10 paso a en_proceso
ERROR: no existe el responsable
--- BUSQUEDAS ---
Pendientes: 4
En proceso: 4
Certificados: 4
De R01: 3
REPORTE DE SOLICITUDES ACADEMICAS
Pendientes: 4
En proceso: 4
Resueltas: 3
Rechazadas: 0
TOTAL: 11
Demora promedio: 0 dias
--- VENCIDAS (mas de 30 dias) ---
BUILD SUCCESSFUL in 3s
2 actionable tasks: 2 executed
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.8.0/userguide/configuration_cache_enabling.

```
2 actionable tasks: 2 executed
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.8.0/userguide/configuration_cache_enabling.html
PS C:\Users\PC\Desktop\Tp-Integrador-Metodologia\Metodologia-II-Trabajo-Integrador\Sistema-Acedemico> ./gradlew run

> Task :run
Codido antes de la refactorizacion
--- ALTA DE SOLICITUDES ---
Solicitud 10 registrada correctamente
Solicitud 11 registrada correctamente
Solicitud 12 registrada correctamente

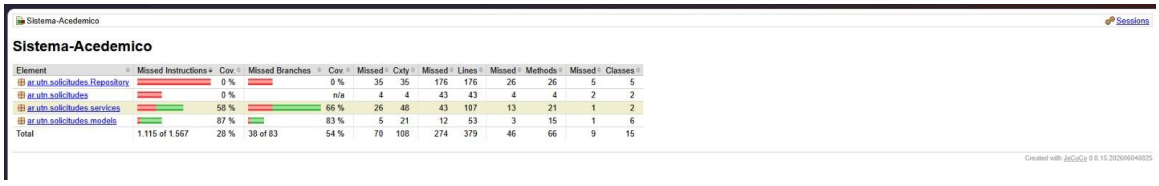
--- CASOS DE ERROR ---
ERROR: falta el legajo
ERROR: email invalido
ERROR: tipo invalido

--- ASIGNACION Y CAMBIO DE ESTADO ---
ERROR: no se puede procesar sin responsable
Solicitud 10 asignada a Laura Gimenez
Solicitud 10 paso a en_proceso
Solicitud 10 paso a resuelta
ERROR: la solicitud ya esta cerrada
Solicitud 11 asignada a Marcos Duarte
Solicitud 11 paso a en_proceso
ERROR: no existe el responsable

--- BUSQUEDAS ---
Pendientes: 4
En proceso: 4
Certificados: 4
De R01: 4

========================================
   REPORTE DE SOLICITUDES ACADEMICAS
========================================
Pendientes:  4
En proceso:  4
Resueltas:   4
Rechazadas:  0
TOTAL:       12
Demora promedio: 0 dias
--- VENCIDAS (mas de 30 dias) ---

BUILD SUCCESSFUL in 2s
2 actionable tasks: 2 executed
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.8.0/userguide/configuration_cache_enabling.html
PS C:\Users\PC\Desktop\Tp-Integrador-Metodologia\Metodologia-II-Trabajo-Integrador\Sistema-Acedemico>
```



Sistema-Acedemico

Sessions

Sistema-Acedemico

Element	Missed Instructions ÷	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
ar.utn.solicitudes.Repository		0 %		0 %	35	35	176	176	26	26	5	5
ar.utn.solicitudes		0 %		n/a	4	4	43	43	4	4	2	2
ar.utn.solicitudes.services		58 %		66 %	26	48	43	107	13	21	1	2
ar.utn.solicitudes.models		87 %		83 %	5	21	12	53	3	15	1	6
Total	1.115 of 1.567	28 %	38 of 83	54 %	70	108	274	379	46	66	9	15

Created with JaCoCo 0.8.15.202606040825

```
BUILD SUCCESSFUL in 4s
5 actionable tasks: 5 executed
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.8.0/userguide/configuration_cache_enabling.html
PS C:\Users\PC\Desktop\Tp-Integrador-Metodologia\Metodologia-II-Trabajo-Integrador\Sistema-Acedemico> .\gradlew.bat clean test

> Task :test

services > Una transicion invalida se rechaza PASSED

services > Con email sin arroba la solicitud se rechaza PASSED

services > Caso de borde: no se permiten mas de 5 solicitudes abiertas PASSED

services > Con responsable asignado, pendiente pasa a en proceso PASSED

services > La prioridad se asigna segun el tipo de solicitud PASSED

services > Al resolverse se registra la fecha de cierre PASSED

services > Una solicitud cerrada no se puede reabrir PASSED

services > Asignar un responsable inexistente falla PASSED

services > Con un tipo inexistente la solicitud se rechaza PASSED

services > Asignar sobre una solicitud inexistente falla PASSED

services > Una solicitud valida se registra y devuelve un id positivo PASSED

services > No se puede poner en proceso una solicitud sin responsable PASSED

services > Sin legajo la solicitud se rechaza PASSED

services > La busqueda por estado devuelve solo las que coinciden PASSED

BUILD SUCCESSFUL in 4s
5 actionable tasks: 5 executed
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.8.0/userguide/configuration_cache_enabling.html
PS C:\Users\PC\Desktop\Tp-Integrador-Metodologia\Metodologia-II-Trabajo-Integrador\Sistema-Acedemico>
```

istema-Acedemico > gradlew.bat